Expression	Description
<%# *ruby code* %>	The Ruby code between the delimiters is ignored (useful for testing).
% *line of ruby code*	A line that starts with a percent is assumed to contain just Ruby code.

You can run ERB from the command line:

erb ‹*options*› ‹*document*›

If *document* is omitted, ERB will read from standard input. The command-line options for ERB are listed in the following table:

Option	Description
-d	Sets $DEBUG to true
-E ext[:int]	Sets the default external/internal encodings
-n	Displays resulting Ruby script (with line numbers)
-r library	Loads the named library
-P	Doesn't do erb processing on lines starting %
-S level	Sets the *safe level*
-T mode	Sets the *trim mode*
-U	Sets default encoding to UTF-8
-v	Enables verbose mode
-x	Displays resulting Ruby script

Let's look at some simple examples. We'll run the ERB executable on the following input:

web/f1.erb
```
<% 99.downto(96) do |number| %>
  <%= number %> bottles of beer...
<% end %>
```

The lines starting with the percent sign simply execute the given Ruby. In this case, it's a loop that iterates the line between them. The sequence <%= number %> in the middle line substitutes in the value of number into the output:

```
$ erb f1.erb
  99 bottles of beer...
  98 bottles of beer...
  97 bottles of beer...
  96 bottles of beer...
```

ERB works by rewriting its input as a Ruby script and then executing that script. You can see the Ruby code ERB generates using the -n or -x option—output here slightly edited for spacing:

```
$ erb -x f1.erb
#coding:UTF-8
_erbout = +'';
99.downto(96) do |number| ;
_erbout.<< "\n".freeze;
_erbout.<< "  ".freeze;
```